TCSS 321 – Discrete Structures

Recursive Algorithms

Recursive Algorithms

Definition: A recursive function is a function that calls itself.

- In order to avoid infinite recursion a recursive function must consist of two cases:
  - A non-recursive terminating case.
  - A recursive case.

Example: The Fibonacci sequence of integers are defined recursively.

$$F(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Recursive Algorithms

$$F(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

- The Fibonacci sequence can be computed by a recursive algorithm.

```
Fibonacci(n)
    If n ≤ 1 return 1.
    Else return Fibonacci(n − 1) + Fibonacci(n − 2).
End
```

Recursive Algorithms

```
Fibonacci(n)
     If n ≤ 1 return 1.
     Else return Fibonacci(n − 1) + Fibonacci(n − 2).
End
```

- How many operations are carried out by this recursive algorithm?
  - In order to analyze the number of operations of a recursive algorithm we need another mathematical tool called a recurrence (or recurrence relation).

Definition: A recurrence (or recurrence relation) is a recursively defined mathematical function.

- The recursive definition of the Fibonacci sequence is an example of a recurrence.

Recursive Algorithms

```
Fibonacci(n)
        If n ≤ 1 return 1.
        Else return Fibonacci(n − 1) + Fibonacci(n − 2).
End
```

- We will define $T(n)$ a recurrence that expresses the number of operations carried out by this recursive algorithm when called on input n.
  - We will consider two cases when $n \leq 1$ (terminating case) and when $n > 1$ (recursive case).
- When $n \leq 1$ we carry out 3 operations (1 method call, 1 comparison, and 1 return).
- When $n > 1$ we will count first only the non-recursive operations.
  - There are 6 operations (1 method call, 1 comparison, 3 arithmetic, and 1 return).
  - In addition we make two recursive calls of size $n - 1$ and $n - 2$.
  - We can express this cost as $T(n-1) + T(n-2)$.

$$T(n) = \begin{cases} 3 & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + 6 & \text{if } n > 1 \end{cases}$$

TCSS 321 – Discrete Structures

Recursive Minimum

Recursive Minimum

Minimum
Input: A sequence of integers $A = (a_1, a_2, \ldots, a_n)$.
Output: $a_i \in A$ such that $a_i \leq a_j$ for all $a_j \in A$.

- Can we write a recursive algorithm to solve this problem?

```
Minimum(A = (a_1, a_2, ⋯, a_n))
        If n = 1 return a_1.
        Else
                Let m = Minimum(A = (a_1, a_2, ⋯, a_{n−1})).
                If a_n < m return a_n
                Else return m.
        End
End
```

Recursive Minimum

```
Minimum(A = (a_1, a_2, ⋯, a_n))
        If n = 1 return a_1.
        Else
                Let m = Minimum(A = (a_1, a_2, ⋯, a_{n−1})).
                If a_n < m return a_n
                Else return m.
        End
End
```

- How many operations are carried out by this recursive algorithm?
  - In the terminating case we carry out 3 operations (1 method call, 1 comparison, and 1 return).
  - In the recursive case we carry out 6 operations in addition to the recursive call (1 method call, 2 comparisons, 1 subtraction, 1 assignment and 1 return).

$$T(n) = \begin{cases} 3 & \text{if } n = 1 \\ T(n-1) + 6 & \text{if } n > 1 \end{cases}$$

Recursive Minimum

$$T(n) = \begin{cases} 3 & \text{if } n = 0 \\ T(n-1) + 6 & \text{if } n > 1 \end{cases}$$

- Let's investigate this recurrence for a few values of n:

$$T(1) = 3$$
$$T(2) = T(1) + 6 = 3 + 6 = 9$$
$$T(3) = T(2) + 6 = 3 + 6 + 6 = 15$$
$$T(4) = T(3) + 6 = 3 + 6 + 6 + 6 = 21$$

- Can we guess an expression for $T(n)$ that is not recursive?

$$T(n) = 3 + 6(n-1) = 6n - 3$$

Recursive Minimum

$$T(n) = \begin{cases} 3 & \text{if } n = 1 \\ T(n-1) + 6 & \text{if } n > 1 \end{cases}$$

Theorem: $T(n) = 6n - 3$

Proof: (by induction on n)

Base case: $(n = 1)$

$$T(1) = 3 = 6(1) - 3$$

Inductive Hypothesis: Assume $T(n-1) = 6(n-1) - 3$**.**

Inductive case: $(n > 1)$

$$\begin{aligned} T(n) &= T(n-1) + 6 \\ &= 6(n-1) - 3 + 6 \\ &= 6n - 6 - 3 + 6 \\ &= 6n - 3 \end{aligned}$$

So by induction we have shown that $T(n) = 6n - 3$ for $n \geq 0$. ■

TCSS 321 – Discrete Structures

Recursive Search

Recursive Search

Search
Input: A sequence of integers $A = (a_1, a_2, \cdots, a_n)$ and a target $t \in \mathbb{Z}$.
Output: i, if $a_i = t$
-1, if no i exists such that $a_i = t$

- Can we write a recursive algorithm to solve this problem?

Search$(A = (a_1, a_2, \cdots, a_n), t)$
If $n = 0$ return -1.
Else If $a_n = t$ return n.
Else return Search$(A = (a_1, a_2, \cdots, a_{n-1}), t)$.
End

Recursive Search

```
Search(A = (a_1, a_2, ⋯, a_n), t)
        If n = 0 return −1.
        Else If a_n = t return n.
        Else return Search(A = (a_1, a_2, ⋯, a_{n−1}), t ).
End
```

- How many operations are carried out by this recursive algorithm?
  - In the terminating case we carry out 3 operations (1 method call, 1 comparison, and 1 return).
  - In the recursive case we carry out 5 operations in addition to the recursive call (1 method call, 2 comparisons, 1 subtraction, and 1 return).

$$T(n) = \begin{cases} 3 & \text{if } n = 0 \\ T(n-1) + 5 & \text{if } n > 0 \end{cases}$$

Recursive Search

$$T(n) = \begin{cases} 3 & \text{if } n = 0 \\ T(n-1) + 5 & \text{if } n > 0 \end{cases}$$

- Let's investigate this recurrence for a few values of n:

$$T(0) = 3$$
$$T(1) = T(0) + 5 = 3 + 5 = 8$$
$$T(2) = T(1) + 5 = 3 + 5 + 5 = 13$$
$$T(3) = T(2) + 5 = 3 + 5 + 5 + 5 = 18$$

- Can we guess an expression for $T(n)$ that is not recursive?

$$T(n) = 3 + 5n$$

Recursive Search

$$T(n) = \begin{cases} 3 & \text{if } n = 0 \\ T(n-1) + 5 & \text{if } n > 0 \end{cases}$$

Theorem: $T(n) = 3 + 5n$

Proof: (by induction on n)

Base case: $(n = 0)$

$$T(0) = 3 = 3 + 5(0)$$

Inductive Hypothesis: Assume $T(n-1) = 3 + 5(n-1)$**.**

Inductive case: $(n > 0)$

$$\begin{aligned} T(n) &= T(n-1) + 5 \\ &= 3 + 5(n-1) + 5 \\ &= 3 + 5n - 5 + 5 \\ &= 3 + 5n \end{aligned}$$

So by induction we have shown that $T(n) = 3 + 5n$ for $n \geq 0$. ■

TCSS 321 – Discrete Structures

Binary Search

Binary Search

Search a Sorted List

Input: A sequence of integers $A = (a_1, a_2, \cdots, a_n)$ such that $a_1 < a_2 < \cdots < a_n$ and a target $t \in \mathbb{Z}$.

Output: i, if $a_i = t$

-1, if no i exists such that $a_i = t$

- Can we write a recursive algorithm to solve this problem?

BinarySearch$(A = (a_a, a_{a+1}, \cdots, a_b), t)$

If $b < a$ return -1.

Else

Let $m = \left\lfloor \frac{a+b}{2} \right\rfloor$.

If $a_m = t$ return m.

Else If $a_m > t$ return BinarySearch$(A = (a_a, a_{a+1}, \cdots, a_{m-1}), t)$.

Else return BinarySearch$(A = (a_{m+1}, a_{m+2}, \cdots, a_b), t)$.

End

End

Binary Search

BinarySearch($A = (a_a, a_{a+1}, \cdots, a_b), t$)
    If $b < a$ return -1.
    Else
        Let $m = \left\lfloor \frac{a+b}{2} \right\rfloor$.
        If $a_m = t$ return m.
        Else If $a_m > t$ return BinarySearch($A = (a_a, a_{a+1}, \cdots, a_{m-1}), t$).
        Else return BinarySearch($A = (a_{m+1}, a_{m+2}, \cdots, a_b), t$).
    End
End

- Can we express the number of operations carried out by this recursive algorithm as a recurrence relation?

$$T(n) = \begin{cases} 3 & \text{if } n < 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 9 & \text{if } n \geq 1 \end{cases}$$

Binary Search

$$T(n) = \begin{cases} 3 & \text{if } n < 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 9 & \text{if } n \geq 1 \end{cases}$$

- Let's investigate this recurrence for a few values of n:

$$T(0) = 3$$
$$T(1) = T(0) + 9 = 3 + 9 = 12$$
$$T(2) = T(1) + 9 = 3 + 9 + 9 = 21$$
$$T(3) = T(1) + 9 = 3 + 9 + 9 = 21$$
$$T(4) = T(2) + 9 = 3 + 9 + 9 + 9 = 30$$
$$T(5) = T(2) + 9 = 3 + 9 + 9 + 9 = 30$$
$$T(6) = T(3) + 9 = 3 + 9 + 9 + 9 = 30$$
$$T(7) = T(3) + 9 = 3 + 9 + 9 + 9 = 30$$
$$T(8 \dots 15) = T(4 \dots 7) + 9 = 3 + 9 + 9 + 9 + 9 = 39$$
$$T(16 \dots 31) = T(8 \dots 15) + 9 = 3 + 9 + 9 + 9 + 9 + 9 = 48$$

- Can we guess an expression for $T(n)$ that is not recursive?

$$T(n) = 3 + 9(\lfloor \log_2 n \rfloor + 1)$$

Binary Search

$$T(n) = \begin{cases} 3 & \text{if } n < 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 9 & \text{if } n \geq 1 \end{cases}$$

- We want to prove:

$$T(n) = 3 + 9(\lfloor \log_2 n \rfloor + 1)$$

- To prove this by induction is challenging because of the floor function. To make the proof easier we will assume, for the moment, that $n = 2^k$ for some $k \in \mathbb{N}$.
- So instead we can rewrite the recurrence as:

$$T(2^k) = \begin{cases} 3 & \text{if } n < 1 \\ T(2^{k-1}) + 9 & \text{if } n \geq 1 \end{cases}$$

- and we will try to prove that:

$$T(2^k) = 3 + 9(\log_2 2^k + 1) = 3 + 9(k + 1)$$

Binary Search

Theorem: $T(2^k) = 3 + 9(k+1)$ for all $k \in \mathbb{N}$.

Proof: (by induction on k)

Base case: ($k = 0$)

$$T(1) = 12 = 3 + 9$$

Inductive Hypothesis: Assume that $T(2^i) = 3 + 9(i+1)$ for $i < k$.

Inductive case: ($k > 0$)

$$\begin{aligned} T(2^k) &= T(2^{k-1}) + 9 \\ &= 3 + 9(k-1+1) + 9 \\ &= 3 + 9k + 9 \\ &= 3 + 9(k+1) \end{aligned}$$

So by induction $T(2^k) = 3 + 9(k+1)$ for all $k \in \mathbb{N}$. ■

- Now assume that $2^k < n < 2^{k+1}$.
  - In this case $\lfloor \log_2 n \rfloor = \log_2 2^k = k$ and $2^{k-1} \leq \left\lfloor \frac{n}{2} \right\rfloor < 2^k$.
  - As a result $T(n) = 3 + 9(\lfloor \log_2 n \rfloor + 1)$.

TCSS 321 – Discrete Structures

Strong Induction

Bounds on Recurrence Relations

- Often exact expressions for a recurrence or summation can be hard to find. However, we are usually only interested in tight upper and lower bounds.
- That is, we only need to show that:

$$a \cdot f(n) \leq T(n) \leq b \cdot f(n)$$

- (for all $n > n_0$) to show that $T(n) \in \Theta\big(f(n)\big)$.

- We can use induction to show upper and lower bounds as well.

- In order to do so with our binary search recurrence we need to introduce strong induction.

Strong Induction

- Thus far we have been using a weaker form of mathematical induction.

Definition: In weak induction the case for n depends only on the case for $n-1$.

Definition: In strong induction the case for n may depend on more than one case, and that case may be something different from $n-1$.

Example: Fibonacci sequence. Since the Fibonacci sequence depends on two sub-cases ($n-1$ and $n-2$) and since one of these is not $n-1$ we would need to use strong induction to prove a tight bound.

Example: Binary search. In binary search we have only one recursive case but it is not $n-1$ (it is $\left\lfloor \frac{n}{2} \right\rfloor$) so we must use strong induction.

Upper Bound

$$T(n) = \begin{cases} 3 & \text{if } n < 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 9 & \text{if } n \geq 1 \end{cases}$$

Theorem: $T(n) \leq b \log_2 n$ for all $n > n_0$.

Proof: (by induction on n)

Base case: $(n = 1)$

$$T(1) = 3 \leq b \log_2 1 = 0$$

- This doesn't work for any choice of b!
- This base case does not hold for $n = 1$ so we must take a different strategy.
- Since we get to choose our n_0 in the definition of $O\big(f(n)\big)$ we don't have to prove the $n = 1$ case.
- Our instinct is to then select $n = 2$ but we must be careful.
  - Note that $\left\lfloor \frac{3}{2} \right\rfloor = 1$. So the case for $n = 3$ depends on the case for $n = 1$ not $n = 2$ so if we only show $n = 2$ this is not sufficient to get induction to work for all n.
  - Instead we need to prove the base case for both $n = 2$ and $n = 3$. (Note: $\left\lfloor \frac{4}{2} \right\rfloor = 2$ so we don't need to prove for $n = 4$.)

Upper Bound

$$T(n) = \begin{cases} 3 & \text{if } n < 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 9 & \text{if } n \geq 1 \end{cases}$$

Theorem: $T(n) \leq b \log_2 n$ for all $n > n_0$.

Proof: (by induction on n)

Base case: ($n = 2,3$)

$$T(2) = 12 \leq b \log_2 2 = b$$

This case is only true if $b \geq 12$.

$$T(3) = 12 \leq b \log_2 3$$

This case is only true if $b \geq \frac{12}{\log_2 3}$.

Inductive hypothesis: Assume $T(k) \leq b \log_2 k$ for all $k < n$.

Inductive case: ($n > 3$)

$$\begin{aligned} T(n) &= T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 9 \\ &\leq b \log_2 \left\lfloor \frac{n}{2} \right\rfloor + 9 \\ &\leq b \log_2 \frac{n}{2} + 9 \\ &= b \log_2 n - b \log_2 2 + 9 \\ &= b \log_2 n - b + 9 \\ &\leq b \log_2 n \end{aligned}$$

This case is only true if $b \geq 9$. So we select $b = 12$ and $n_0 = 2$. By induction $T(n) \leq b \log_2 n$ for all $n > n_0$. ■

Lower Bound

$$T(n) = \begin{cases} 3 & \text{if } n < 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 9 & \text{if } n \geq 1 \end{cases}$$

Theorem: $T(n) \geq a \log_2 n$ for all $n > n_0$.

Proof: (by induction on n)

Base case: $(n = 1)$

$$T(1) \geq a \log_2 1 = 0$$

This case is true for all a.

Inductive hypothesis: Assume $T(k) \geq a \log_2 k$ for all $k < n$.

Inductive case: $(n > 1)$

$$\begin{aligned} T(n) &= T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 9 \\ &\geq a \log_2 \left\lfloor \frac{n}{2} \right\rfloor + 9 \\ &\geq a \log_2 \frac{n}{4} + 9 \\ &= a \log_2 n - a \log_2 4 + 9 \\ &= a \log_2 n - 2a + 9 \\ &\geq a \log_2 n \end{aligned}$$

This case is true only if $a \leq \frac{9}{2}$. So we select $a = \frac{9}{2}$ and $n_0 = 1$. So by induction $T(n) \geq a \log_2 n$ for all $n > n_0$. ■

Floor and Ceiling

- In these proofs we have had to approximate away the floor (or ceiling) function. To do this we have used the following rules:

$$x - 1 \leq \lfloor x \rfloor \leq x$$
$$x \leq \lceil x \rceil \leq x + 1$$

- These approximations work most of the time, but it can be problematic inside a function like a $\log n$ or $\sqrt{n}$.
- So another useful approximation for the floor is:

$$\frac{n}{4} \leq \left\lfloor \frac{n}{2} \right\rfloor$$

- What values of n is this true for?

$$n \geq 2$$

Tight Bounds

- Our two theorems have shown:

$$T(n) \in O(\log_2 n)$$
$$T(n) \in \Omega(\log_2 n)$$

- Together they establish:

$$T(n) \in \Theta(\log_2 n)$$

- We can establish this even if we cannot find a $f(n)$ such that:

$$T(n) = f(n)$$